Data Dictionary: Feedback Intelligence Platform

Status	Approved
Version	1.7
Date	2026-11-01
Owner	Benjamin Rose
Related	Requirements, ADR 0009: data layers, ADR 0013: bodies excluded from history

This document defines every table and column the system stores, with its type and purpose, and the data contract that controls what may leave the pipeline. It implements the layered design in ADR 0009: data moves from **raw** to **cleaned** to **reporting**, and only the reporting layer is published to the dashboard.

1. Data types used

Type	What it holds	Why it's used here
`tinyint`	Whole numbers 0 to 255	Tiny counts and rankings; 1 byte
`smallint`	Whole numbers up to about 32,000	Small IDs and counts; 2 bytes
`int`	Whole numbers up to about 2.1 billion	Issue numbers and most counts; 4 bytes
`bigint`	Whole numbers up to about 9.2 quintillion	GitHub run IDs, which already exceed `int`; 8 bytes
`decimal(7,1)`	A number with one decimal place, up to 999,999.9	Durations in days, like 3.5 days to close; exact, so medians and percentages don't drift
`bit`	0 or 1	True/false flags
`varchar(n)`	Text up to *n* characters, basic characters only	Fixed-vocabulary values (states, categories) and URLs, which never contain special characters
`nvarchar(n)`	Text up to *n* characters (at most 4,000), any language and emoji	Anything people write, since issues come from all over the world
`nvarchar(max)`	Text of any length, up to 2 GB	Raw issue bodies, which have no size limit on GitHub
`binary(32)`	Exactly 32 bytes of raw data	A SHA-256 hash, which is always 32 bytes

Type	What it holds	Why it's used here
`date`	A calendar date, no time	Week start dates, where a time of day would be meaningless
`datetime2(0)`	Date and time to the second	All timestamps, always in UTC so time zones never cause confusion

2. Schemas

Schema	What it holds and why
`raw`	Issues exactly as fetched, including unredacted text. Kept separate so it can be emptied every run and is never saved or published.
`cleaned`	Redacted issues, repositories, AI results, and weekly digests. This is the system's long-term memory and the only issue data saved as history.
`reporting`	Aggregates and dashboard-ready data, rebuilt each run. The only layer the publisher reads from, so nothing else can leak out.
`ops`	Records about the pipeline itself: runs and ingestion progress. Kept apart from issue data because it describes the system, not the content.

3. Raw layer

`raw.issue`

One row per issue fetched in the current run. It's emptied at the start of every run, so it only ever holds one run's worth of unredacted data, and that data disappears with the runner.

Column	Type	What it does and why
`repo_id`	`smallint`	Identifies which tracked repository the issue came from. Uses a small numeric ID instead of the full name so it's compact and joins quickly.
`issue_number`	`int`	The issue's number as shown on GitHub (e.g., #4521). Together with `repo_id` it uniquely identifies an issue, and forms the **primary key**.
`title`	`nvarchar(1000)`	The issue title exactly as written, **unredacted**. 1,000 characters comfortably exceeds GitHub's practical title length.
`body`	`nvarchar(max)`	The full issue text, **unredacted**. Uses `max` because GitHub bodies can be very long, often pasted logs.
`state`	`varchar(10)`	Whether the issue is `open` or `closed`. Needed for counting open issues and time-to-close.

Column	Type	What it does and why
`state_reason`	`varchar(20)`	Why it closed (`completed` , `not_planned` , `duplicate`). Lets metrics separate fixed issues from rejected ones.
`labels`	`nvarchar(2000)`	The maintainers' own GitHub labels, stored as a JSON list. Kept as useful context and for comparison with AI categories.
`created_at`	`datetime2(0)`	When the issue was opened. Drives "new issues this week" and trend charts.
`updated_at`	`datetime2(0)`	When the issue last changed. Drives incremental loading: only issues updated since the last successful run are fetched.
`closed_at`	`datetime2(0)`	When the issue was closed, or empty if still open. Used to calculate time to close.
`html_url`	`varchar(300)`	The link to the issue on GitHub. Powers the "view original" link (US-08).
`fetched_at`	`datetime2(0)`	When our pipeline fetched the issue. Useful for debugging and data lineage.

Deliberately excluded: the author's username. GitHub provides it, but the ingestor never stores it, because data you don't collect can't leak (FR-PRV-02). Pull requests are filtered out before insert (FR-ING-03).

4. Cleaned layer

`cleaned.repository`

The list of tracked repositories. A small lookup table so every other table can refer to a repository by a short numeric ID.

Column	Type	What it does and why
`repo_id`	`smallint`	**Primary key.** The short numeric ID other tables use. `smallint` allows thousands of repositories, far more than needed.
`owner`	`varchar(100)`	The GitHub account that owns the repository (e.g., `microsoft`). Needed to build API calls.
`name`	`varchar(100)`	The repository name (e.g., `winget-cli`). Combined with `owner` , it's what the GitHub API and dashboard display.
`onboarded_at`	`datetime2(0)`	When tracking began. Explains why a repository's history starts on a certain date.

`cleaned.issue`

Every issue ever seen, with personal information removed. This is the core of the system's history. Issue bodies are held here **only during a run** and are never saved to history (ADR 0013).

Column	Type	What it does and why
`repo_id`	`smallint`	Which repository. First half of the **primary key**.
`issue_number`	`int`	GitHub issue number. Second half of the **primary key**.
`title_redacted`	`nvarchar(1000)`	The title after Presidio has replaced personal information with placeholders. Safe to store, show, and send to the AI.
`body_redacted`	`nvarchar(4000)`	The first 4,000 characters of the redacted body, present **only during a run** for issues fetched in that run. Used as AI input, then dropped before history is saved, so restored issues have this empty (ADR 0013).
`body_length`	`int`	The original body's length before truncation. Saved to history so the "summarize if longer than 500 characters" rule still works without the body (FR-AI-02).
`body_truncated`	`bit`	1 if the body was cut at 4,000 characters. Makes the truncation visible to the AI step and to anyone debugging.
`content_hash`	`binary(32)`	A SHA-256 fingerprint of the redacted title and full redacted body. If the text changes, the fingerprint changes, which tells the AI step whether reprocessing is needed.
`redaction_count`	`smallint`	How many items Presidio replaced in this issue. Useful for monitoring redaction and spotting unusual issues.
`state`	`varchar(10)`	`open` or `closed`, copied from raw. Needed for metrics.
`state_reason`	`varchar(20)`	Why the issue closed, copied from raw. Separates fixed issues from rejected ones in metrics.
`labels`	`nvarchar(2000)`	Maintainer labels as JSON, copied from raw. Labels are chosen by maintainers, so they contain no personal data.
`created_at`	`datetime2(0)`	When the issue was opened. Indexed because weekly metrics filter on it constantly.
`updated_at`	`datetime2(0)`	When the issue last changed. Indexed because incremental loading and upserts depend on it.
`closed_at`	`datetime2(0)`	When the issue closed. Used for time-to-close metrics.
`html_url`	`varchar(300)`	Link to the original issue. Shown on the dashboard.
`first_seen_run_id`	`bigint`	The pipeline run that first stored this issue. Lets you trace exactly when data entered the system.

Column	Type	What it does and why
`last_changed_run_id`	`bigint`	The run that last changed this issue. Helps debug what a specific run did.

`cleaned.issue_enrichment`

AI results for each piece of issue text. It also serves as the **cache**: results are keyed by the text's fingerprint, so unchanged text is never sent to the AI twice (FR-AI-03).

Column	Type	What it does and why
`content_hash`	`binary(32)`	Which text this result belongs to; matches `cleaned.issue.content_hash`. First half of the **primary key**.
`prompt_version`	`varchar(20)`	Which version of the prompt produced the result. Second half of the **primary key**, so improving a prompt triggers fresh results instead of reusing old ones.
`status`	`varchar(15)`	`classified` or `unclassified`. Records when the AI's answer failed validation, rather than guessing (FR-AI-04).
`category`	`varchar(20)`	One of the categories listed in the configuration file (currently bug, feature_request, question, docs, performance). Powers filtering and trends (US-05).
`severity`	`varchar(10)`	One of the severities listed in the configuration file (currently low, medium, high, critical). Lets engineering leads prioritize (US-05).
`severity_reason`	`nvarchar(300)`	One line explaining the severity. Makes the AI's judgment checkable instead of a black box.
`summary`	`nvarchar(1000)`	A 1–3 sentence summary, empty when the body is 500 characters or shorter. Saves readers from long logs (US-07).
`model`	`varchar(100)`	Which AI model produced the result. Needed if results change after switching models or providers.
`enriched_at`	`datetime2(0)`	When the result was created. Useful for audits and debugging.
`run_id`	`bigint`	Which pipeline run created it. Traces AI usage back to specific runs.

`cleaned.weekly_digest`

Stores each weekly digest permanently. It lives in the cleaned layer, not reporting, because past digests must be kept (FR-INS-05) and the reporting layer is rebuilt each run.

Column	Type	What it does and why
`week_start`	`date`	**Primary key.** The Monday the digest's week begins. One digest per week.

Column	Type	What it does and why
`digest_text`	`nvarchar(3000)`	The digest itself, under 300 words (about 2,000 characters), with room to spare. Written by the AI from redacted data only.
`model`	`varchar(100)`	Which AI model wrote it. Needed to explain differences if the model changes.
`prompt_version`	`varchar(20)`	Which prompt version produced it. Lets digests be compared fairly over time.
`generated_at`	`datetime2(0)`	When it was written.
`run_id`	`bigint`	Which pipeline run created it. Traces it back to that run's logs.

5. Operational tables

`ops.run`

One row per pipeline run. It's the system's logbook: it feeds the pipeline health page (US-11) and helps diagnose failures.

Column	Type	What it does and why
`run_id`	`bigint`	**Primary key.** The GitHub Actions run ID, so every row links directly to that run's logs.
`trigger`	`varchar(10)`	`schedule` or `manual`. Distinguishes automatic daily runs from ones started by hand.
`backfill_from`	`datetime2(0)`	First issue creation date a backfill re-fetched (midnight UTC), empty for normal runs. Records what a backfill covered (US-14).
`backfill_to`	`datetime2(0)`	Last creation date the backfill re-fetched (midnight UTC; the whole day is included), empty for normal runs.
`started_at`	`datetime2(0)`	When the run began. With `finished_at`, gives run duration (NFR-PERF-03).
`finished_at`	`datetime2(0)`	When the run ended, empty while running.
`status`	`varchar(10)`	`running`, `succeeded`, or `failed`. The headline for the health page and the success-rate target (NFR-REL-01).
`issues_fetched`	`int`	How many issues came from GitHub this run. A sudden drop to zero can signal a problem.
`issues_rejected`	`int`	How many fetched records failed validation and were skipped (FR-ING-05). If more than 10% are rejected, the run fails, since that suggests an API change (risk R7).

Column	Type	What it does and why
`issues_redacted`	`int`	How many issues passed through Presidio. Should normally equal `issues_fetched`.
`ai_requests`	`smallint`	How many AI requests were sent. Tracked against the daily free limit (FR-AI-05).
`ai_cache_hits`	`int`	How many issues reused cached AI results. Shows how much the cache saves.
`issues_unclassified`	`int`	How many issues the AI couldn't classify. A spike can mean a prompt or model problem.
`failed_step`	`varchar(50)`	Which step failed, empty on success. Included in alert issues (FR-OPS-02).
`error_message`	`nvarchar(2000)`	The error text, empty on success. Truncated, and must contain only system details, never issue content, because it's saved to the public history branch and may appear in alerts. SQL Server errors that can quote data values are stored as their error number only.

`ops.watermark`

How far ingestion has reached for each repository. It's what makes loading incremental, fetching only what's new.

Column	Type	What it does and why
`repo_id`	`smallint`	**Primary key.** One watermark per repository.
`last_updated_at`	`datetime2(0)`	The latest issue `updated_at` from the last fully successful run. The next run fetches issues updated at or after this time (FR-ING-02); re-fetching one at exactly this time is harmless because issues are upserted.
`last_success_run_id`	`bigint`	Which run set the watermark. Makes it easy to check where ingestion resumed.

Key rule: the watermark only moves forward when a run fully succeeds. If a run fails halfway, the next run starts from the old watermark and fetches the missed issues again. Duplicates can't happen, because issues are upserted (FR-DAT-03).

6. Reporting layer

Every reporting table is **rebuilt from the cleaned and ops layers on each run**, so it can never drift out of sync. The publisher reads only from this layer. Wherever a table has a `repository` column, it holds either a full repository name or `all` for all repositories combined.

`reporting.issue`

One row per issue, shaped for the issue explorer (US-06, US-08), with the latest AI results attached. It's the largest published table.

Column	Type	What it does and why
`repository`	`varchar(201)`	Full name like `microsoft/winget-cli`. Included as text rather than an ID so published files and CSV exports make sense on their own.
`issue_number`	`int`	GitHub issue number. With `repository`, forms the **primary key**.
`title`	`nvarchar(1000)`	The **redacted** title. The only issue text published apart from the AI summary.
`state`	`varchar(10)`	`open` or `closed`. Used for filtering and counts.
`state_reason`	`varchar(20)`	Why it closed, such as `completed` or `not_planned`. Gives context in the explorer.
`created_at`	`datetime2(0)`	When opened. Drives date filters.
`updated_at`	`datetime2(0)`	When last changed. Lets users sort by recent activity.
`closed_at`	`datetime2(0)`	When closed, empty if open.
`days_to_close`	`decimal(7,1)`	Days between opening and closing, empty if open. Calculated once here so every page and metric uses the same value.
`category`	`varchar(20)`	AI category. Empty unless classified.
`severity`	`varchar(10)`	AI severity. Empty unless classified.
`severity_reason`	`nvarchar(300)`	The AI's one-line justification. Lets readers judge the label.
`summary`	`nvarchar(1000)`	AI summary, empty for short issues. Searchable along with the title (US-06).
`ai_status`	`varchar(15)`	`classified`, `unclassified`, or `pending`. `pending` means not processed yet, for example because the daily AI limit was reached, so the dashboard can show it honestly instead of as a gap.
`html_url`	`varchar(300)`	Link to the original on GitHub (US-08).

Deliberately excluded: labels, body text, content hash, redaction counts, and run IDs. No page needs them, so under data minimization (NFR-PRIV-02) they aren't published.

`reporting.overview`

The numbers on the Overview page (US-01): the past 7 days and the 7 days before, per repository and for all repositories combined.

Column	Type	What it does and why
`repository`	`varchar(201)`	A repository name, or `all` for the combined view.
`period`	`varchar(10)`	`current` (last 7 days) or `previous` (the 7 days before). Two rows per repository let the dashboard show week-over-week change.
`period_start`	`datetime2(0)`	Start of the 7-day window. Shown so readers know exactly what's counted.
`period_end`	`datetime2(0)`	End of the window.
`new_issues`	`int`	Issues opened in the window.
`closed_issues`	`int`	Issues closed in the window.
`bug_count`	`int`	New issues classified as bugs. Stored as a count, not a percentage, so the dashboard calculates the percentage the same way everywhere. Empty until AI classification exists (M4).
`critical_count`	`int`	New issues classified as critical. Empty until AI classification exists (M4).
`median_days_to_close`	`decimal(7,1)`	Median days to close for issues closed in the window. The median isn't skewed by a few very old issues the way an average would be. Empty when no issues closed in the window.

Why an `all` row is stored instead of calculated: medians can't be combined. The median of three repositories' medians isn't the true median of all issues, so the combined figure is calculated from the underlying issues in SQL.

`reporting.weekly_metrics`

Weekly totals for the Trends page (US-02). One row per repository per week, plus an `all` row per week.

Column	Type	What it does and why
`repository`	`varchar(201)`	A repository name, or `all`.
`week_start`	`date`	The Monday the week begins. Weeks start on Monday (the ISO standard) so they're consistent everywhere. Only complete weeks are included, starting with the first full week of a repository's initial history window; the current week is covered by the Overview instead. Published as a timestamp at midnight UTC.
`new_issues`	`int`	Issues opened that week. Drives the volume chart.

Column	Type	What it does and why
`closed_issues`	`int`	Issues closed that week. Shows whether the backlog is growing or shrinking.
`critical_count`	`int`	Critical issues opened that week. Highlights severity spikes. Empty until AI classification exists (M4).
`median_days_to_close`	`decimal(7,1)`	Median days to close for issues closed that week. Shows whether fixes are getting faster.

`reporting.weekly_category`

How each week's new issues split across categories, for the category-mix chart (US-02).

Column	Type	What it does and why
`repository`	`varchar(201)`	A repository name, or `all`.
`week_start`	`date`	The Monday the week begins.
`category`	`varchar(20)`	A category, or `unclassified` or `pending`. Including these keeps the totals honest instead of silently dropping issues.
`issue_count`	`int`	New issues in that category that week.

`reporting.topic`

Emerging topics for the Emerging topics page (US-03, built in M5).

Column	Type	What it does and why
`topic_id`	`int`	**Primary key.** Identifies the topic within this run's results; topics are recalculated daily.
`repository`	`varchar(201)`	Which repository the topic belongs to. Topics are found per repository, since themes rarely cross products.
`name`	`nvarchar(100)`	A short AI-written name, like "Terminal crashes after update."
`summary`	`nvarchar(1000)`	An AI-written paragraph describing the theme, based on redacted issue text only.
`issues_current`	`int`	Issues in the topic during the past 7 days. Must be at least 3 to appear (FR-INS-03).
`issues_previous`	`int`	Issues in the topic during the 7 days before. The baseline for growth.
`growth_pct`	`decimal(7,1)`	Percentage growth from the previous to the current window. Rankings use this.

Column	Type	What it does and why
`rank`	`tinyint`	Position on the page, 1 being the fastest-growing.
`generated_at`	`datetime2(0)`	When the topic was calculated.

`reporting.topic_issue`

Which issues belong to each topic, so selecting a topic lists its issues (US-03).

Column	Type	What it does and why
`topic_id`	`int`	The topic. Part of the **primary key**.
`repository`	`varchar(201)`	The issue's repository. Part of the **primary key**.
`issue_number`	`int`	The issue. Part of the **primary key**; joins to `reporting.issue` for title and link.

`reporting.digest`

A **view** (a saved query rather than a stored table) that exposes `cleaned.weekly_digest` to the publisher (US-04). It publishes only `week_start`, `digest_text`, and `generated_at`; the model and prompt version stay internal.

`reporting.run_health`

The last 30 pipeline runs, for the Pipeline health page (US-11).

Column	Type	What it does and why
`run_id`	`bigint`	The GitHub Actions run ID. Viewers can open the public run logs from it.
`trigger`	`varchar(10)`	`schedule` or `manual`.
`started_at`	`datetime2(0)`	When the run began.
`duration_seconds`	`int`	How long it took. Calculated here so the dashboard doesn't need to.
`status`	`varchar(10)`	`succeeded` or `failed`.
`issues_fetched`	`int`	Issues pulled from GitHub.
`ai_requests`	`smallint`	AI requests used.
`ai_cache_hits`	`int`	Issues served from cache.
`issues_unclassified`	`int`	Issues the AI couldn't classify.

Column	Type	What it does and why
`failed_step`	`varchar(50)`	Which step failed, if any.

Deliberately excluded: `error_message`. Error details can reveal internal system information, so they go only into alert issues and run logs, never onto the dashboard.

`reporting.site_info`

A single row of facts about the current data, published as a tiny file the dashboard loads first.

Column	Type	What it does and why
`data_updated_at`	`datetime2(0)`	The moment the data reflects: the start of the run that produced it. Every number in one run is measured against this same moment. Shown on every page (FR-DSH-05).
`last_run_id`	`bigint`	The run that produced the data.
`repositories`	`nvarchar(1000)`	The tracked repositories, as a JSON list. Fills the repository filter.
`ai_model`	`varchar(100)`	The AI model in use. Shown alongside AI-generated labels for transparency (FR-DSH-06). This and the next three columns are empty until AI enrichment exists (M4).
`prompt_version`	`varchar(20)`	The current prompt version.
`ai_daily_limit`	`smallint`	The free daily AI request limit. With the latest usage, shows how close the system is to its limit (US-11).
`ai_requests_today`	`smallint`	AI requests used in the latest run.

7. Data contract

The data contract is the **complete list of what may ever leave the pipeline**. Anything not listed is forbidden. An **automated test** reads the tables below directly from this document and compares every output file and its columns against them, so adding a file or field requires changing this contract through a pull request (NFR-PRIV-02). Keep the table formats below unchanged, since the test parses them.

Output 1: published dashboard files (GitHub Pages)

File	Format	Source	Fields
`site_info.json`	JSON	`reporting.site_info`	All columns
`overview.json`	JSON	`reporting.overview`	All columns

File	Format	Source	Fields
`issues.parquet`	Parquet	`reporting.issue`	All columns
`weekly_metrics.parquet`	Parquet	`reporting.weekly_metrics`	All columns
`weekly_category.parquet`	Parquet	`reporting.weekly_category`	All columns
`topics.parquet`	Parquet	`reporting.topic`	All columns
`topic_issues.parquet`	Parquet	`reporting.topic_issue`	All columns
`digests.parquet`	Parquet	`reporting.digest`	`week_start`, `digest_text`, `generated_at`
`run_health.parquet`	Parquet	`reporting.run_health`	All columns

The two small JSON files let the dashboard show the Overview page immediately while DuckDB-WASM loads in the background, which addresses the load-time risk in ADR 0008.

Output 2: history files (public `data` branch)

File	Source	Fields
`repositories.parquet`	`cleaned.repository`	All columns
`issues.parquet`	`cleaned.issue`	All columns **except** `body_redacted`
`enrichment.parquet`	`cleaned.issue_enrichment`	All columns
`digests.parquet`	`cleaned.weekly_digest`	All columns
`runs.parquet`	`ops.run`	All columns
`watermarks.parquet`	`ops.watermark`	All columns

Rules for both outputs

1. Nothing from the `raw` schema is ever written to any file.
2. No issue body text is ever written to any file.
3. No GitHub usernames exist anywhere in the system.
4. Error messages appear only in history and alert issues, never on the dashboard, and never contain issue content.
5. Both outputs are publicly readable, so both are held to the same privacy standard.

8. Design notes

- **Issue bodies never leave a run.** They're used as AI input during the run that fetched them, then dropped. Reprocessing older issues, for example after a prompt improvement, re-fetches them from GitHub through a backfill (ADR 0013).
- **Bodies are capped at 4,000 characters.** That's the largest fixed `nvarchar` size, enough context for the AI, and it keeps AI requests small enough to batch several issues together. The original length is kept so the summary rule still works.
- **Only titles and AI summaries are published.** Readers get the full discussion through the link to GitHub.
- **Counts are stored, not percentages.** Percentages and changes are calculated in one place, the dashboard, from stored counts, so they can't disagree between pages.
- **Timestamps are always UTC.** Conversion to local time happens only in the dashboard, for display.

Revision history

Version	Date	Change
1.7	2026-11-01	Clarified that the backfill range is by issue creation date, with both days included.
1.6	2026-11-01	`ops.run.error_message` keeps only the error number for SQL Server errors that can quote data values.
1.5	2026-11-01	Noted which weeks `reporting.weekly_metrics` covers and how `week_start` is published. The automated data contract test now reads section 7 directly.
1.4	2026-11-01	Clarified that ingestion resumes at, not after, the watermark.
1.3	2026-10-31	Clarified that `reporting.site_info.data_updated_at` is the run's start time.
1.2	2026-10-24	Noted which `reporting.overview` and `reporting.site_info` columns are empty until AI enrichment exists, and that the median is empty when nothing closed.
1.1	2026-10-18	Added `ops.run.issues_rejected`; categories and severities now come from configuration (see design).
1.0	2026-10-14	Added reporting layer, `cleaned.weekly_digest`, and data contract. Excluded bodies from history (ADR 0013). Corrected body cap to 4,000 characters (`nvarchar(8000)` is not a valid SQL Server type).
0.1	2026-10-14	Draft: schemas, raw, cleaned, and operational layers